

Практические задачи по теме "Память в C++" с подробными комментариями

Задача 1: Анализ размещения переменных в памяти

```
#include <iostream>
#include <vector>
using namespace std;

// Глобальные переменные - статическая память
int global_var = 100;
const char* global_str = "Global String";

void analyzeMemoryLocations() {
    /*
    ЗАДАЧА: Проанализировать, где находятся разные переменные

    ВОПРОСЫ:
    1. Какие переменные находятся в стеке, а какие в статической памяти?
    2. Почему адреса глобальных переменных отличаются от локальных?
    3. Что происходит с памятью при выходе из функции?
    */

    cout << "=== ЗАДАЧА 1: Анализ размещения переменных ===" << endl;
    cout << "===== " << endl;

    // 1. Локальные переменные (стек)
    int local_int = 42;
    double local_array[3] = {1.1, 2.2, 3.3};

    // 2. Статическая локальная переменная
    static int static_local = 999;
```

```

// 3. Динамическая память (куча)
int* heap_int = new int(777);
vector<int>* heap_vector = new vector<int>{1, 2, 3};

cout << "\nАДРЕСА ПЕРЕМЕННЫХ:" << endl;
cout << "-----" << endl;

// Глобальные переменные (статика)
cout << "Глобальные (статика):" << endl;
cout << "  global_var:  " << &global_var
      << "  значение: " << global_var << endl;
cout << "  global_str:  " << (void*)global_str
      << "  значение: \"" << global_str << "\"" << endl;

// Локальные переменные (стек)
cout << "\nЛокальные (стек):" << endl;
cout << "  local_int:    " << &local_int
      << "  значение: " << local_int << endl;
cout << "  local_array: " << local_array
      << "  первый элемент: " << local_array[0] << endl;

// Статическая локальная
cout << "\nСтатическая локальная:" << endl;
cout << "  static_local: " << &static_local
      << "  значение: " << static_local << endl;

// Динамические переменные (куча)
cout << "\nДинамические (куча):" << endl;
cout << "  heap_int:      " << heap_int
      << "  значение: " << *heap_int << endl;
cout << "  Адрес указателя heap_int в стеке: " << &heap_i

// ВОПРОС ДЛЯ АНАЛИЗА:
cout << "\n--- ВОПРОСЫ ДЛЯ АНАЛИЗА ---" << endl;
cout << "1. Почему адрес global_var сильно отличается от
cout << "  Ответ: Они в разных сегментах памяти (статика

cout << "\n2. Почему &heap_int и heap_int разные?" << endl;
cout << "  Ответ: &heap_int - адрес указателя в стеке,"
cout << "  heap_int - адрес данных в куче, на которые он

cout << "\n3. Что произойдет с памятью при выходе из функ

```

```

cout << "    Ответ: Локальные переменные (стек) уничтожатся;
cout << "    статическая переменная сохранится," << endl;
cout << "    динамическая память останется выделенной (утечка)";

// Важная часть: освобождение памяти
cout << "\n--- ОСВОБОЖДЕНИЕ ПАМЯТИ ---" << endl;
delete heap_int;
delete heap_vector;
heap_int = nullptr;

cout << "Память освобождена (без утечек!)" << endl;
}

// Дополнительный анализ
void memoryDistanceAnalysis() {
    cout << "\n\n=== ДОПОЛНИТЕЛЬНЫЙ АНАЛИЗ: Расстояния между ;

    // Создаем несколько локальных переменных
    int a = 1;
    int b = 2;
    int c = 3;

    cout << "\nЛокальные переменные в стеке:" << endl;
    cout << "a: " << &a << " значение: " << a << endl;
    cout << "b: " << &b << " значение: " << b << endl;
    cout << "c: " << &c << " значение: " << c << endl;

    // Вычисляем расстояния
    cout << "\nРасстояния между адресами:" << endl;
    cout << "От a до b: " << (char*)&b - (char*)&a << " байт"
    cout << "От b до c: " << (char*)&c - (char*)&b << " байт"
    cout << "От a до c: " << (char*)&c - (char*)&a << " байт"

    // ВОПРОС:
    cout << "\n--- ВОПРОС ---" << endl;
    cout << "Почему расстояния не равны 4 байтам (размер int)?"
    cout << "Ответ: Компилятор может добавлять выравнивание (padding)
    cout << "для оптимизации доступа к памяти." << endl;
}

```

Задача 2: Утечки памяти и их обнаружение

```
#include <iostream>
#include <cstring>
using namespace std;

// Счетчик "утечек" для демонстрации
static int leak_counter = 0;

void demonstrateMemoryLeaks() {
    /*
    ЗАДАЧА: Показать различные виды утечек памяти.

    ВОПРОСЫ:
    1. Как возникают утечки памяти?
    2. Как их обнаружить?
    3. Как предотвратить?
    */

    cout << "\n\n=== ЗАДАЧА 2: Утечки памяти ===" << endl;
    cout << "===== " << endl;

    cout << "Текущее количество 'утечек': " << leak_counter << endl;

    // 1. ПРОСТАЯ УТЕЧКА: забыли delete
    cout << "\n1. Простая утечка:" << endl;
    int* simple_leak = new int(100);
    cout << "Выделена память: " << simple_leak
        << " значение: " << *simple_leak << endl;
    // ЗАБЫЛИ: delete simple_leak;
    leak_counter++;

    // 2. УТЕЧКА В ЦИКЛЕ: выделение без освобождения в цикле
    cout << "\n2. Утечка в цикле:" << endl;
    for (int i = 0; i < 5; i++) {
        int* cycle_leak = new int(i * 10);
        cout << "Цикл " << i << ": выделено " << cycle_leak << endl;
        // УТЕЧКА: каждый new без delete
        leak_counter++;
    }

    // 3. УТЕЧКА ПРИ ДОСРОЧНОМ ВОЗВРАТЕ
```

```
cout << "\n3. Утечка при досрочном возврате:" << endl;
int* early_return_leak = new int(999);

if (true) { // Условие всегда истинно
    cout << "Досрочный возврат!" << endl;
    // return; // Если бы был return - утечка!
}

delete early_return_leak; // Сделали правильно
cout << "Память освобождена (утечки нет)" << endl;

// 4. УТЕЧКА В ВЕТКАХ УСЛОВИЙ
cout << "\n4. Утечка в ветках условий:" << endl;
bool condition = true;
int* conditional_leak = nullptr;

if (condition) {
    conditional_leak = new int(111);
    cout << "Выделили в if: " << *conditional_leak << endl;
} else {
    conditional_leak = new int(222);
    cout << "Выделили в else: " << *conditional_leak << endl;
}

// ОПАСНОСТЬ: можно забыть delete в одной из веток
delete conditional_leak; // Не забыли!

// 5. УТЕЧКА ПРИ ИСКЛЮЧЕНИЯХ
cout << "\n5. Утечка при исключениях:" << endl;
int* exception_leak = new int(333);

try {
    cout << "Пытаемся что-то сделать..." << endl;
    // throw runtime_error("Исключение!"); // Раскомментируйте
    delete exception_leak; // Если исключение - не выполнится
} catch (...) {
    cout << "Поймали исключение, но память не освободили!" << endl;
    // delete exception_leak; // Нужно здесь!
    leak_counter++;
}

// РЕШЕНИЯ:
cout << "\n--- РЕШЕНИЯ ПРОБЛЕМ УТЕЧЕК ---" << endl;
```

```

// Решение 1: RAII (деструктор освобождает)
class SafeInt {
private:
    int* data;
public:
    SafeInt(int value) {
        data = new int(value);
        cout << "SafeInt создан: " << *data << endl;
    }

    ~SafeInt() {
        delete data;
        cout << "SafeInt уничтожен, память освобождена" << endl;
    }
};

{
    cout << "\nРешение 1: RAII класс" << endl;
    SafeInt safe(555); // Память выделена
    // При выходе из блока деструктор автоматически освободит
}

// Решение 2: умные указатели
cout << "\nРешение 2: умные указатели" << endl;
{
    unique_ptr<int> smart_ptr = make_unique<int>(666);
    cout << "Умный указатель: " << *smart_ptr << endl;
    // Автоматически удалится при выходе из блока
}

// Решение 3: контейнеры STL
cout << "\nРешение 3: контейнеры STL" << endl;
vector<int> numbers = {1, 2, 3, 4, 5};
numbers.push_back(6); // Автоматическое управление памятью

cout << "\nИтоговое количество 'утечек': " << leak_counter << endl;
cout << "Эти утечки существуют только в демонстрационных программах" << endl;
cout << "В реальной программе они привели бы к росту потребления памяти" << endl;
}

```

Задача 3: Анализ производительности стека и кучи

```
#include <iostream>
#include <chrono>
#include <vector>
using namespace std;
using namespace std::chrono;

void performanceComparison() {
    /*
    ЗАДАЧА: Сравнить производительность стека и кучи.

    ВОПРОСЫ:
    1. Что быстрее: выделение в стеке или куче?
    2. Почему есть разница в скорости?
    3. Когда стоит использовать каждый подход?
    */

    cout << "\n\n=== ЗАДАЧА 3: Производительность стека vs кучи\n\n";
    cout << "=====

    const int ITERATIONS = 100000;
    const int ARRAY_SIZE = 100;

    // 1. ТЕСТ: выделение в стеке
    cout << "\n1. ТЕСТ: Выделение в стеке" << endl;

    auto start_stack = high_resolution_clock::now();

    for (int i = 0; i < ITERATIONS; i++) {
        // Выделяем массив в стеке
        int stack_array[ARRAY_SIZE];

        // Используем массив (чтобы компилятор не оптимизировал)
        for (int j = 0; j < ARRAY_SIZE; j++) {
            stack_array[j] = j;
        }

        // Массив автоматически уничтожается при выходе из области видимости
    }
}
```

```

auto end_stack = high_resolution_clock::now();
auto duration_stack = duration_cast<microseconds>(end_stack - start_stack);

cout << "Время стека: " << duration_stack.count() << " мкс" << endl;
cout << "Среднее время на итерацию: "
    << duration_stack.count() / ITERATIONS << " мкс" << endl;

// 2. ТЕСТ: выделение в куче
cout << "\n2. ТЕСТ: Выделение в куче" << endl;

auto start_heap = high_resolution_clock::now();

for (int i = 0; i < ITERATIONS; i++) {
    // Выделяем массив в куче
    int* heap_array = new int[ARRAY_SIZE];

    // Используем массив
    for (int j = 0; j < ARRAY_SIZE; j++) {
        heap_array[j] = j;
    }

    // ОБЯЗАТЕЛЬНО освобождаем память
    delete[] heap_array;
}

auto end_heap = high_resolution_clock::now();
auto duration_heap = duration_cast<microseconds>(end_heap - start_heap);

cout << "Время кучи: " << duration_heap.count() << " мкс" << endl;
cout << "Среднее время на итерацию: "
    << duration_heap.count() / ITERATIONS << " мкс" << endl;

// 3. АНАЛИЗ РЕЗУЛЬТАТОВ
cout << "\n--- АНАЛИЗ РЕЗУЛЬТАТОВ ---" << endl;

double ratio = (double)duration_heap.count() / duration_stack.count();
cout << "Куча медленнее стека в " << ratio << " раз" << endl;

cout << "\nПОЧЕМУ ТАК ПРОИСХОДИТ:" << endl;
cout << "1. Стек: простое перемещение указателя стека (очень быстро)" << endl;
cout << "2. Куча: поиск свободного блока, работа с менеджером памяти" << endl;
cout << "3. Куча: дополнительная фрагментация и метаданные" << endl;

```



```

// 4. ТЕСТ: большое выделение (где куча имеет смысл)
cout << "\n3. ТЕСТ: Большое выделение (1MB)" << endl;

const int BIG_SIZE = 1000000; // 1 миллион int ~ 4MB

// Стек: НЕЛЬЗЯ! Слишком большой массив
// int huge_stack[BIG_SIZE]; // Stack Overflow!

// Куча: можно
auto start_big_heap = high_resolution_clock::now();

int* big_heap_array = new int[BIG_SIZE];
for (int i = 0; i < BIG_SIZE; i++) {
    big_heap_array[i] = i;
}
delete[] big_heap_array;

auto end_big_heap = high_resolution_clock::now();
auto duration_big_heap = duration_cast<microseconds>(end_

cout << "Выделение 4MB в куче: " << duration_big_heap.count() << endl;

// 5. ВЫВОДЫ И РЕКОМЕНДАЦИИ
cout << "\n--- РЕКОМЕНДАЦИИ ---" << endl;
cout << "ИСПОЛЬЗУЙТЕ СТЕК, когда:" << endl;
cout << "• Небольшие объекты (< 1KB)" << endl;
cout << "• Временные данные (в пределах функции)" << endl;
cout << "• Требуется максимальная производительность" << endl;

cout << "\nИСПОЛЬЗУЙТЕ КУЧУ, когда:" << endl;
cout << "• Большие объекты (> 1KB)" << endl;
cout << "• Данные должны жить дольше функции" << endl;
cout << "• Размер известен только во время выполнения" << endl;
cout << "• Нужны сложные структуры данных" << endl;

// 6. ПРАКТИЧЕСКИЙ ПРИМЕР
cout << "\n--- ПРАКТИЧЕСКИЙ ПРИМЕР ---" << endl;

// ПЛОХО: выделение мелких объектов в куче
cout << "ПЛОХОЙ КОД (утечки и медленно):" << endl;
for (int i = 0; i < 1000; i++) {
    int* bad = new int(i); // Мелкий объект в куче
}

```

```

        // Используем...
        // delete bad; // Часто забывают
    }

    // ХОРОШО: стек для мелких объектов
    cout << "\nХОРОШИЙ КОД (быстро и безопасно):" << endl;
    for (int i = 0; i < 1000; i++) {
        int good = i; // В стеке - быстро и автоматически
        // Используем...
    }
}

```

Задача 4: Работа с двумерными массивами в памяти

```

#include <iostream>
#include <iomanip>
using namespace std;

void twoDimensionalArrays() {
    /*
    ЗАДАЧА: Понять, как двумерные массивы располагаются в памяти

    ВОПРОСЫ:
    1. Как устроен двумерный массив в памяти?
    2. В чем разница между статическим и динамическим 2D массивом?
    3. Как эффективно работать с большими матрицами?
    */

    cout << "\n\n=== ЗАДАЧА 4: Двумерные массивы в памяти ===\n";
    cout << "===== " << endl;

    const int ROWS = 3;
    const int COLS = 4;

    // 1. СТАТИЧЕСКИЙ 2D МАССИВ (в стеке)
    cout << "\n1. Статический 2D массив (стек):" << endl;

    int static_matrix[ROWS][COLS] = {

```

```

        {1, 2, 3, 4},
        {5, 6, 7, 8},
        {9, 10, 11, 12}
    };

    cout << "Матрица " << ROWS << "x" << COLS << ":" << endl;
    for (int i = 0; i < ROWS; i++) {
        for (int j = 0; j < COLS; j++) {
            cout << setw(3) << static_matrix[i][j] << " ";
        }
        cout << endl;
    }

    cout << "\nАдреса элементов (показывает непрерывность):" << endl;
    cout << "static_matrix[0][0]: " << &static_matrix[0][0] << endl;
    cout << "static_matrix[0][1]: " << &static_matrix[0][1]
        << " (" << (char*)&static_matrix[0][1] - (char*)&static_matrix[0][0] << ")<< endl;
    cout << "static_matrix[1][0]: " << &static_matrix[1][0]
        << " (" << (char*)&static_matrix[1][0] - (char*)&static_matrix[0][0] << ")<< endl;

    // ВОПРОС: Почему static_matrix[1][0] не сразу после static_matrix[0][3]?
    cout << "\n--- ВОПРОС ---" << endl;
    cout << "Матрица хранится ПОСТРОЧНО в непрерывной памяти!" << endl;
    cout << "[0][0] [0][1] [0][2] [0][3] [1][0] [1][1] ..." << endl;

    // 2. ДИНАМИЧЕСКИЙ 2D МАССИВ (в куче) - НЕПРАВИЛЬНЫЙ СПОСОБ
    cout << "\n\n2. Динамический 2D массив (неправильный способ)" << endl;

    // Создаем массив указателей на строки
    int** dynamic_matrix_bad = new int*[ROWS];

    // Каждая строка - отдельный массив
    for (int i = 0; i < ROWS; i++) {
        dynamic_matrix_bad[i] = new int[COLS];
        for (int j = 0; j < COLS; j++) {
            dynamic_matrix_bad[i][j] = i * COLS + j + 1;
        }
    }

    cout << "Создан массив указателей и " << ROWS << " отдельных массивов" << endl;

    // Проблема: данные НЕ непрерывны в памяти!
    cout << "\nАдреса строк (показывает НЕ непрерывность):" << endl;

```

```

for (int i = 0; i < ROWS; i++) {
    cout << "Строка " << i << ": " << dynamic_matrix_bad[i]
}

// Проблема: медленный доступ и больше накладных расходов
cout << "\nПРОБЛЕМЫ этого подхода:" << endl;
cout << "1. Данные не непрерывны (кэш-промахи)" << endl;
cout << "2. Много мелких выделений памяти" << endl;
cout << "3. Сложное освобождение" << endl;

// Освобождаем (в обратном порядке!)
for (int i = 0; i < ROWS; i++) {
    delete[] dynamic_matrix_bad[i];
}
delete[] dynamic_matrix_bad;

// 3. ДИНАМИЧЕСКИЙ 2D МАССИВ (в куче) - ПРАВИЛЬНЫЙ СПОСОБ
cout << "\n\n3. Динамический 2D массив (правильный способ

// Выделяем один большой непрерывный блок
int* dynamic_matrix_good = new int[ROWS * COLS];

// Заполняем как одномерный массив
for (int i = 0; i < ROWS; i++) {
    for (int j = 0; j < COLS; j++) {
        dynamic_matrix_good[i * COLS + j] = i * COLS + j
    }
}

cout << "Выделен один непрерывный блок " << ROWS * COLS *
    << " байт" << endl;
cout << "Адрес начала: " << dynamic_matrix_good << endl;

// Доступ через формулу: элемент[i][j] = data[i * COLS + j]
cout << "\nДоступ к элементам:" << endl;
cout << "dynamic_matrix_good[1][2] = "
    << dynamic_matrix_good[1 * COLS + 2] << endl;

// Преимущества:
cout << "\nПРЕИМУЩЕСТВА этого подхода:" << endl;
cout << "1. Непрерывная память (лучше для кэша)" << endl;
cout << "2. Одно выделение/освобождение" << endl;
cout << "3. Быстрее и эффективнее" << endl;

```

```

delete[] dynamic_matrix_good;

// 4. ПРАКТИЧЕСКОЕ ЗАДАНИЕ: реализовать класс Matrix
cout << "\n\n4. ПРАКТИЧЕСКОЕ ЗАДАНИЕ: класс Matrix" << endl;

class Matrix {
private:
    int rows, cols;
    int* data; // Один непрерывный блок

public:
    Matrix(int r, int c) : rows(r), cols(c) {
        data = new int[rows * cols]; // Одно выделение
        cout << "Матрица " << rows << "x" << cols
              << " создана (" << rows * cols * sizeof(int)
    }

    ~Matrix() {
        delete[] data; // Одно освобождение
        cout << "Память матрицы освобождена" << endl;
    }

    // Доступ к элементу
    int& at(int i, int j) {
        return data[i * cols + j];
    }

    // Заполнение
    void fill(int value) {
        for (int i = 0; i < rows * cols; i++) {
            data[i] = value;
        }
    }

    // Вывод
    void print() const {
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                cout << setw(4) << data[i * cols + j];
            }
            cout << endl;
        }
    }
}

```

```

    }
};

cout << "\nСоздаем и используем матрицу 4x5:" << endl;
{
    Matrix m(4, 5);
    m.fill(0);

    // Заполняем диагональ
    for (int i = 0; i < 4; i++) {
        m.at(i, i) = i + 1;
    }

    m.print();
    // При выходе из блока деструктор освободит память
}

cout << "\nМатрица автоматически уничтожена (RAII)" << endl;
}

```

Задача 5: Отладка проблем с памятью

```

#include <iostream>
#include <cstring>
using namespace std;

// Глобальный счетчик для отслеживания выделений
static int allocation_count = 0;
static int deallocation_count = 0;

// Перегружаем new и delete для отслеживания
void* operator new(size_t size) {
    allocation_count++;
    cout << "[NEW] Выделено " << size << " байт. Всего выделено "
        << allocation_count << endl;
    return malloc(size);
}

void operator delete(void* ptr) noexcept {
    deallocation_count++;
}

```

```

    cout << "[DELETE] Освобождена память. Всего освобождений:
        << deallocation_count << endl;
    free(ptr);
}

void operator delete[](void* ptr) noexcept {
    deallocation_count++;
    cout << "[DELETE[]] Освобожден массив. Всего освобождений
        << deallocation_count << endl;
    free(ptr);
}

void debugMemoryIssues() {
    /*
    ЗАДАЧА: Найти и исправить проблемы с памятью.

    ВОПРОСЫ:
    1. Где происходит утечка памяти?
    2. Где возможен доступ к неинициализированной памяти?
    3. Как правильно исправить код?
    */

    cout << "\n\n=== ЗАДАЧА 5: Отладка проблем с памятью ==="
    cout << "===== " << endl;

    allocation_count = 0;
    deallocation_count = 0;

    cout << "Начальное состояние: выделений=" << allocation_count
        << ", освобождений=" << deallocation_count << endl;

    // КОД С ОШИБКАМИ (найдите и исправьте!)
    cout << "\n--- КОД С ОШИБКАМИ ---" << endl;

    // ОШИБКА 1: Неинициализированный указатель
    cout << "\n1. Неинициализированный указатель:" << endl;
    {
        int* ptr1; // ОШИБКА: не инициализирован
        // *ptr1 = 42; // ОПАСНО: указатель указывает "в никто"

        // ИСПРАВЛЕНИЕ:
        int* ptr1_fixed = nullptr; // Инициализируем
        // или лучше: int* ptr1_fixed = new int(42);
    }
}

```

```
}

// ОШИБКА 2: Утечка памяти
cout << "\n2. Утечка памяти:" << endl;
{
    int* ptr2 = new int(100);
    cout << "Создали: " << *ptr2 << endl;
    // ЗАБЫЛИ: delete ptr2; // УТЕЧКА!

    // ИСПРАВЛЕНИЕ:
    delete ptr2; // Не забываем!
    ptr2 = nullptr;
}

// ОШИБКА 3: Двойное удаление
cout << "\n3. Двойное удаление:" << endl;
{
    int* ptr3 = new int(200);
    delete ptr3; // Первое удаление - ОК

    // delete ptr3; // ОШИБКА: второе удаление того же уи

    // ИСПРАВЛЕНИЕ:
    ptr3 = nullptr; // После delete устанавливаем в null
    // Тогда повторный delete безопасен (delete nullptr не
}

// ОШИБКА 4: Доступ после удаления
cout << "\n4. Доступ после удаления:" << endl;
{
    int* ptr4 = new int(300);
    delete ptr4; // Освободили память

    // *ptr4 = 400; // ОПАСНО: доступ к освобожденной паи

    // ИСПРАВЛЕНИЕ:
    ptr4 = nullptr; // Обнуляем указатель
    // Теперь попытка доступа будет явной ошибкой
}

// ОШИБКА 5: Несоответствие new[] и delete
cout << "\n5. Несоответствие new[] и delete:" << endl;
{
```



```

int* arr = new int[5]; // Выделили массив

// delete arr; // ОШИБКА: нужно delete[] arr

// ИСПРАВЛЕНИЕ:
delete[] arr; // Правильно для массивов
}

// ОШИБКА 6: Переполнение буфера
cout << "\n6. Переполнение буфера:" << endl;
{
    char buffer[10];
    // strcpy(buffer, "Очень длинная строка, которая не п

    // ИСПРАВЛЕНИЕ:
    strncpy(buffer, "Короткая", sizeof(buffer) - 1);
    buffer[sizeof(buffer) - 1] = '\0'; // Гарантируем за

}

// ПРАКТИЧЕСКОЕ ЗАДАНИЕ: найти все ошибки
cout << "\n--- ПРАКТИЧЕСКОЕ ЗАДАНИЕ ---" << endl;
cout << "Найдите все ошибки в следующем коде:" << endl;

cout << "\nКОД ДЛЯ АНАЛИЗА:" << endl;
cout << "{" << endl;
cout << "    int* p1 = new int(10);" << endl;
cout << "    int* p2 = p1;" << endl;
cout << "    " << endl;
cout << "    delete p1;" << endl;
cout << "    " << endl;
cout << "    *p2 = 20; // ОШИБКА 1: доступ через висячий
cout << "    " << endl;
cout << "    char* str = new char[50];" << endl;
cout << "    strcpy(str, \"Hello\");" << endl;
cout << "    delete str; // ОШИБКА 2: нужно delete[]" << endl;
cout << "    " << endl;
cout << "    int* arr = new int[100];" << endl;
cout << "    // Забыли delete[] arr // ОШИБКА 3: утечка"
cout << "}" << endl;

// АНАЛИЗ ТЕКУЩЕГО СОСТОЯНИЯ
cout << "\n--- АНАЛИЗ ТЕКУЩЕГО СОСТОЯНИЯ ---" << endl;
cout << "Выделений: " << allocation_count << endl;

```

```

cout << "Освобождений: " << deallocation_count << endl;

if (allocation_count > deallocation_count) {
    cout << "ОБНАРУЖЕНА УТЕЧКА ПАМЯТИ!" << endl;
    cout << "Не освобождено: " << allocation_count - deallocation_count
        << " блоков памяти" << endl;
} else if (allocation_count == deallocation_count) {
    cout << "Память управляется правильно (без утечек)" << endl;
} else {
    cout << "Странная ситуация: освобождений больше чем выделено" << endl;
}

// ВОССТАНОВЛЕНИЕ ОРИГИНАЛЬНЫХ new/delete
cout << "\nВосстанавливаем оригинальные операторы new/delete" << endl;
}

```

Задача 6: Создание простого менеджера памяти

```

#include <iostream>
#include <vector>
#include <cassert>
using namespace std;

void simpleMemoryManager() {
    /*
    ЗАДАЧА: Создать простой менеджер памяти для понимания, как работает
    менеджер памяти.

    ВОПРОСЫ:
    1. Как менеджер памяти отслеживает свободные и занятые блоки?
    2. Что такое фрагментация памяти?
    3. Как работает выделение и освобождение?
    */

    cout << "\n\n=== ЗАДАЧА 6: Простой менеджер памяти ===" << endl;
    cout << "===== " << endl;

    // 1. МОДЕЛЬ ПРОСТОГО МЕНЕДЖЕРА ПАМЯТИ
    cout << "\n1. Модель простого менеджера памяти:" << endl;

    const int MEMORY_SIZE = 100; // "Память" в 100 байт

```

```

char memory_pool[MEMORY_SIZE]; // Наш "пул" памяти

// Структура для метаданных блока
struct BlockHeader {
    size_t size;           // Размер блока данных (без заголовка)
    bool is_free;          // Свободен ли блок
    BlockHeader* next;     // Следующий блок
};

// Инициализируем первый блок на всю память
BlockHeader* first_block = reinterpret_cast<BlockHeader*>
first_block->size = MEMORY_SIZE - sizeof(BlockHeader);
first_block->is_free = true;
first_block->next = nullptr;

cout << "Создан пул памяти: " << MEMORY_SIZE << " байт" << endl;
cout << "Первый блок: " << first_block->size << " байт данных" << endl;
cout << "Заголовок блока: " << sizeof(BlockHeader) << " байт" << endl;

// 2. ВИЗУАЛИЗАЦИЯ ПАМЯТИ
cout << "\n2. Визуализация памяти:" << endl;

auto print_memory = [&]() {
    cout << "\n┌────────────────── МЕНЕДЖЕР ПАМЯТИ ───────────────────┐" << endl;
    cout << "| Пул: ";
    for (int i = 0; i < 50; i++) {
        if (i < MEMORY_SIZE) {
            cout << (i % 10);
        } else {
            cout << " ";
        }
    }
    cout << " |" << endl;

    BlockHeader* current = first_block;
    int position = 0;

    while (current) {
        cout << "| ";
        for (int i = 0; i < position; i++) cout << " ";

        if (current->is_free) {
            cout << "[СВОБОДНО " << current->size << "б]"

```

```

        } else {
            cout << "[ЗАНЯТО  " << current->size << "б]";
        }

        position += sizeof(BlockHeader) + current->size;
        current = current->next;
    }

    cout << "\n|_____|" .
};

print_memory();

// 3. ПРОСТАЯ ИМИТАЦИЯ ВЫДЕЛЕНИЯ
cout << "\n3. Имитация выделения памяти:" << endl;

// "Выделяем" 20 байт
cout << "\nЗапрос: выделить 20 байт" << endl;

BlockHeader* current = first_block;
while (current && !(current->is_free && current->size >= :
    current = current->next;
}

if (current) {
    // Нашли подходящий блок
    current->is_free = false;
    cout << "Выделено 20 байт по адресу: "
        << (void*)(current + 1) << endl;
}

print_memory();

// 4. ПРОБЛЕМА: ФРАГМЕНТАЦИЯ ПАМЯТИ
cout << "\n4. Проблема: фрагментация памяти" << endl;

cout << "\nПредставьте ситуацию:" << endl;
cout << "Память: [ЗАНЯТО 10б][СВОБОДНО 20б][ЗАНЯТО 10б]" .
cout << "Запрос: выделить 25 байт" << endl;
cout << "Проблема: есть 20 свободных байт, но они разбиты
cout << "Это и есть ФРАГМЕНТАЦИЯ." << endl;

// 5. РЕАЛЬНЫЙ ПРИМЕР С new/delete

```

```

cout << "\n5. Реальный пример: что происходит при new/delete

cout << "\nШАГИ ПРИ new int(42):" << endl;
cout << "1. Менеджер памяти ищет свободный блок ≥ sizeof(int)" << endl;
cout << "2. Если находит, помечает как занятый" << endl;
cout << "3. Если не находит, запрашивает у ОС больше памяти" << endl;
cout << "4. Возвращает указатель на данные (после заголовка)" << endl;

cout << "\nШАГИ ПРИ delete ptr:" << endl;
cout << "1. По указателю находит заголовок блока" << endl;
cout << "2. Помечает блок как свободный" << endl;
cout << "3. Пытается объединить соседние свободные блоки" << endl;

// 6. ПРАКТИЧЕСКОЕ ЗАДАНИЕ: написать свой malloc/free
cout << "\n6. Практическое задание: простой аллокатор" << endl;

class SimpleAllocator {
private:
    struct Block {
        size_t size;
        bool free;
        Block* next;
    };

    static const size_t POOL_SIZE = 1024;
    char pool[POOL_SIZE];
    Block* first;

public:
    SimpleAllocator() {
        // Инициализируем пул
        first = reinterpret_cast<Block*>(pool);
        first->size = POOL_SIZE - sizeof(Block);
        first->free = true;
        first->next = nullptr;

        cout << "Аллокатор создан с пулом " << POOL_SIZE << endl;
    }

    void* allocate(size_t size) {
        Block* current = first;

        while (current) {

```

```

        if (current->free && current->size >= size) {
            // Нашли подходящий блок
            current->free = false;

            // Можно разделить блок, если осталось место
            if (current->size > size + sizeof(Block)) {
                Block* new_block = reinterpret_cast<Block*>(
                    reinterpret_cast<char*>(current +
                        current->size - size));

                new_block->size = current->size - size;
                new_block->free = true;
                new_block->next = current->next;

                current->size = size;
                current->next = new_block;
            }

            cout << "Выделено " << size << " байт" << endl;
            return reinterpret_cast<void*>(current + sizeof(Block));
        }
        current = current->next;
    }

    cout << "Недостаточно памяти для выделения " << size << " байт" << endl;
    return nullptr;
}

void deallocate(void* ptr) {
    if (!ptr) return;

    Block* block = reinterpret_cast<Block*>(ptr) - 1;
    block->free = true;

    cout << "Освобождено " << block->size << " байт" << endl;

    // Попытка объединить со следующим свободным блоком
    if (block->next && block->next->free) {
        block->size += sizeof(Block) + block->next->size;
        block->next = block->next->next;
    }
}

void print_status() const {

```

```

        cout << "\nСтатус аллокатора:" << endl;
        Block* current = first;
        int block_num = 1;

        while (current) {
            cout << "Блок " << block_num++ << ": ";
            cout << (current->free ? "СВОБОДНО " : "ЗАНЯТО ");
            cout << current->size << " байт" << endl;
            current = current->next;
        }
    };

    cout << "\nТестируем аллокатор:" << endl;
    SimpleAllocator allocator;

    void* ptr1 = allocator.allocate(100);
    void* ptr2 = allocator.allocate(200);
    void* ptr3 = allocator.allocate(50);

    allocator.print_status();

    allocator.deallocate(ptr2);
    cout << "\nПосле освобождения ptr2:" << endl;
    allocator.print_status();

    void* ptr4 = allocator.allocate(150);
    cout << "\nПосле запроса 150 байт:" << endl;
    allocator.print_status();

    // 7. ВЫВОДЫ И ЗАКЛЮЧЕНИЕ
    cout << "\n--- ВЫВОДЫ ---" << endl;
    cout << "1. Менеджер памяти отслеживает блоки через метады";
    cout << "2. Фрагментация возникает из-за множества мелких";
    cout << "3. Аллокация требует поиска подходящего блока" << endl;
    cout << "4. Освобождение может объединять соседние свободные";
    cout << "5. Реальные аллокаторы сложнее (кеши, пулы, стра";
}

```

Главная программа с меню выбора задач

```
int main() {
    cout << "
    cout << "|| ПРАКТИЧЕСКИЕ ЗАДАЧИ ПО ПАМЯТИ C++ ||"
    cout << "

    int choice;

    do {
        cout << "\n\nВЫБЕРИТЕ ЗАДАЧУ:" << endl;
        cout << "1. Анализ размещения переменных в памяти" << endl;
        cout << "2. Утечки памяти и их обнаружение" << endl;
        cout << "3. Производительность стека vs кучи" << endl;
        cout << "4. Двумерные массивы в памяти" << endl;
        cout << "5. Отладка проблем с памятью" << endl;
        cout << "6. Создание простого менеджера памяти" << endl;
        cout << "0. Выход" << endl;
        cout << "\nВаш выбор: ";
        cin >> choice;

        switch (choice) {
            case 1:
                analyzeMemoryLocations();
                memoryDistanceAnalysis();
                break;
            case 2:
                demonstrateMemoryLeaks();
                break;
            case 3:
                performanceComparison();
                break;
            case 4:
                twoDimensionalArrays();
                break;
            case 5:
                debugMemoryIssues();
                break;
            case 6:
                simpleMemoryManager();
                break;
            case 0:
                cout << "Выход из программы" << endl;
                break;
        }
    } while (choice != 0);
}
```



```

        default:
            cout << "Неверный выбор!" << endl;
    }

    if (choice != 0) {
        cout << "\n\nНажмите Enter для продолжения...";
        cin.ignore();
        cin.get();
    }

} while (choice != 0);

cout << "\n★ ПРАКТИЧЕСКИЕ ВЫВОДЫ ★" << endl;
cout << "_____" << endl;
cout << "1. Всегда понимайте, где находятся ваши данные" << endl;
cout << "2. Следите за утечками памяти" << endl;
cout << "3. Выбирайте стек для мелких/временных данных" << endl;
cout << "4. Используйте кучу для больших/долгоживущих данных" << endl;
cout << "5. Пользуйтесь умными указателями и RAII" << endl;
cout << "6. Понимайте работу менеджера памяти" << endl;

return 0;
}

```

Ключевые аспекты, которые покрывают задачи:

Задача 1: Понимание размещения переменных

- Анализ адресов переменных
- Различение стека, кучи, статической памяти
- Понимание областей видимости

Задача 2: Обнаружение утечек памяти

- Типичные причины утечек
- Методы обнаружения
- Профилактика (RAII, умные указатели)

Задача 3: Анализ производительности

- Измерение времени выделения
- Понимание причин разницы в скорости
- Практические рекомендации

Задача 4: Работа с многомерными массивами

- Понимание расположения в памяти
- Эффективные структуры данных
- Создание собственных классов

Задача 5: Отладка проблем

- Типичные ошибки работы с памятью
- Методы отладки
- Практические задания на поиск ошибок

Задача 6: Внутреннее устройство

- Принципы работы менеджера памяти
- Фрагментация и ее причины
- Создание простого аллокатора

Полный разбор: Анализ памяти и отладка с Valgrind в C++

Часть 1: Установка и основы Valgrind

1.1 Установка Valgrind на Ubuntu

Откройте терминал и выполните:

1. Обновление списка пакетов

```
sudo apt update
```

2. Установка Valgrind

```
sudo apt install valgrind -y
```

3. Проверка установки

```
valgrind --version
```

1.2 Что такое Valgrind?

Valgrind - это инструмент для отладки проблем с памятью:

- **Утечки памяти** (memory leaks)
- **Неинициализированная память** (uninitialized memory)
- **Некорректное использование памяти** (invalid memory access)
- **Неправильное освобождение памяти** (invalid free)

Часть 2: Простые примеры проблем с памятью

Пример 1: Простая утечка памяти

```
// Сохраните как example1_leak.cpp
#include <iostream>

int main() {
    std::cout << "=== Пример 1: Простая утечка памяти ===\n" << std::endl;

    // ВЫДЕЛЯЕМ память в куче
    int* number = new int(42);

    std::cout << "Выделили память для int: " << *number << std::endl;

    // ЗАБЫВАЕМ освободить память - УТЕЧКА!
    // delete number; // ЭТОЙ СТРОЧКИ НЕТ!
```

```

    std::cout << "Завершение программы...\n";
    std::cout << "Память для 'number' НЕ освобождена!" << std::endl;

    return 0;
}

```

Компиляция и запуск Valgrind:

```

# Компилируем с отладочной информацией
g++ -g -o example1 example1_leak.cpp

# Запускаем с Valgrind
valgrind --leak-check=full ./example1

```

Пример 2: Несколько утечек

```

// Сохраните как example2_multiple_leaks.cpp
#include <iostream>

void create_leak() {
    // Утечка в функции
    int* leak1 = new int(100);
    std::cout << "В функции создана утечка: " << *leak1 << std::endl;
    // Забыли delete
}

int main() {
    std::cout << "=== Пример 2: Несколько утечек ===\n" << std::endl;

    // Утечка 1
    int* main_leak = new int(200);
    std::cout << "Утечка в main: " << *main_leak << std::endl;

    // Утечка 2
    double* array_leak = new double[10];
    std::cout << "Утечка массива из 10 double" << std::endl;

    // Утечка 3 (из функции)
    create_leak();
}

```

```

// Утечка 4 (в цикле)
for(int i = 0; i < 5; i++) {
    int* loop_leak = new int(i);
    std::cout << "Утечка в цикле #" << i << ": " << *loop_
    // Каждый раз новая утечка!
}

// Освобождаем ТОЛЬКО одну переменную
delete main_leak;

std::cout << "\nЗавершение с 6 утечками памяти!" << std::endl;
return 0;
}

```

Запуск Valgrind:

```

g++ -g -o example2 example2_multiple_leaks.cpp
valgrind --leak-check=full ./example2

```

Часть 3: Разбор вывода Valgrind

3.1 Понимание вывода Valgrind

Давайте создадим программу и разберем вывод Valgrind:

```

// Сохраните как example3_analyze.cpp
#include <iostream>
#include <cstring>

class Student {
private:
    char* name;
    int age;

public:
    Student(const char* studentName, int studentAge) {
        name = new char[strlen(studentName) + 1];
        strcpy(name, studentName);
    }
};

```

```
        age = studentAge;
        std::cout << "Создан студент: " << name << std::endl;
    }

    // ПЛОХО: нет деструктора - утечка памяти для name!
    // ~Student() { delete[] name; } // Этот деструктор нужен

    void display() const {
        std::cout << "Студент: " << name << ", Возраст: " << ;
    }
};

int* createArray(int size) {
    int* arr = new int[size];
    for(int i = 0; i < size; i++) {
        arr[i] = i * 10;
    }
    return arr;
}

void processData() {
    // Утечка: создали массив, но не удалили
    int* data = createArray(5);
    std::cout << "Массив создан: ";
    for(int i = 0; i < 5; i++) {
        std::cout << data[i] << " ";
    }
    std::cout << std::endl;
    // Забыли: delete[] data;
}

int main() {
    std::cout << "=== Пример 3: Анализ вывода Valgrind ===\n"

    // Утечка 1: объект класса без деструктора
    Student* student = new Student("Иван Иванов", 20);
    student->display();
    // Забыли: delete student;

    // Утечка 2: массив в функции
    processData();

    // Утечка 3: простой указатель
```

```

int* single = new int(777);
std::cout << "Значение: " << *single << std::endl;
// Забыли: delete single;

std::cout << "\nПрограмма завершена с утечками памяти!" <<

return 0;
}

```

Компиляция и запуск:

```

g++ -g -o example3 example3_analyze.cpp
valgrind --leak-check=full --show-leak-kinds=all ./example3

```

3.2 Разбор типичного вывода Valgrind:

```

==12345== HEAP SUMMARY:
==12345==      in use at exit: 52 bytes in 4 blocks
==12345==    total heap usage: 5 allocs, 1 frees, 73 bytes allocated
==12345==
==12345== 4 bytes in 1 blocks are definitely lost in loss record 1 of 2
==12345==    at 0x483BE63: operator new(unsigned long) (vg_replace_malloc.c:296)
==12345==    by 0x1092C5: main (example3.cpp:46)
==12345==
==12345== 20 bytes in 1 blocks are definitely lost in loss record 2 of 2
==12345==    at 0x483BE63: operator new(unsigned long) (vg_replace_malloc.c:296)
==12345==    by 0x1091B9: Student::Student(char const*, int)
==12345==    by 0x10928A: main (example3.cpp:40)

```

Ключевые элементы вывода:

1. **HEAP SUMMARY** - общая статистика:

- `in use at exit`: память не освобождена при выходе
- `total heap usage`: всего выделений/освобождений

2. **"definitely lost"** - точно утерянная память

3. **"indirectly lost"** - косвенно утерянная (через другие объекты)

4. **"possibly lost"** - возможно утерянная (опасные указатели)

5. **Номера строк** - где произошло выделение памяти

Часть 4: Распространенные ошибки и их отладка

4.1 Двойное освобождение (double free)

```
// Сохраните как example4_double_free.cpp
#include <iostream>

int main() {
    std::cout << "=== Пример 4: Двойное освобождение ===\n" << '\n';

    int* ptr = new int(100);
    std::cout << "Выделили: " << *ptr << std::endl;

    // Первое освобождение - ОК
    delete ptr;
    std::cout << "Первый delete выполнен" << std::endl;

    // Второе освобождение - ОШИБКА!
    delete ptr; // Double free!
    std::cout << "Второй delete - ОШИБКА!" << std::endl;

    return 0;
}
```

Запуск Valgrind:

```
g++ -g -o example4 example4_double_free.cpp
valgrind ./example4
```

Вывод Valgrind покажет:

```
Invalid free() / delete / delete[] / realloc()
```


4.2 Доступ к освобожденной памяти (use after free)

```
// Сохраните как example5_use_after_free.cpp
#include <iostream>

int main() {
    std::cout << "=== Пример 5: Доступ после освобождения ==="

    int* arr = new int[5];

    // Заполняем массив
    for(int i = 0; i < 5; i++) {
        arr[i] = i * 10;
    }

    // Освобождаем память
    delete[] arr;
    std::cout << "Память освобождена" << std::endl;

    // ОПАСНО: доступ к освобожденной памяти!
    std::cout << "Пытаемся прочитать arr[2]: " << arr[2] << std::endl;
    std::cout << "Это use-after-free ошибка!" << std::endl;

    // Еще хуже: запись в освобожденную память
    arr[0] = 999; // Неопределенное поведение!

    return 0;
}
```



4.3 Неинициализированная память

```
// Сохраните как example6_uninitialized.cpp
#include <iostream>

int main() {
    std::cout << "=== Пример 6: Неинициализированная память ==="

    // Выделяем память, но не инициализируем
    int* values = new int[10];
```

```
// Используем неинициализированную память
int sum = 0;
for(int i = 0; i < 10; i++) {
    sum += values[i]; // Значения не определены!
}

std::cout << "Сумма неинициализированных значений: " << sum;
std::cout << "(значение может быть любым!)" << std::endl;

// Правильно: инициализируем память
for(int i = 0; i < 10; i++) {
    values[i] = 0;
}

delete[] values;
return 0;
}
```

Запуск с проверкой неинициализированной памяти:

```
g++ -g -o example6 example6_uninitialized.cpp
valgrind --track-origins=yes ./example6
```

Ключевой флаг: `--track-origins=yes` - показывает, откуда взялись неинициализированные значения.



Часть 5: Практическое задание с исправлением ошибок

5.1 Программа с множеством ошибок

```
// Сохраните как example7_buggy.cpp
#include <iostream>
#include <cstring>

class DynamicArray {
private:
    int* data;
```

```

    int size;

public:
    DynamicArray(int sz) : size(sz) {
        data = new int[size]; // Выделение памяти
        // ПРОБЛЕМА: не инициализировали массив
    }

    ~DynamicArray() {
        delete[] data; // Освобождение памяти
    }

    void fill(int value) {
        for(int i = 0; i <= size; i++) { // ОШИБКА: выход за
            data[i] = value;
        }
    }

    int get(int index) const {
        return data[index]; // Нет проверки границ
    }
};

void processArray() {
    DynamicArray* arr = new DynamicArray(5);
    arr->fill(10);

    // Используем массив
    for(int i = 0; i < 5; i++) {
        std::cout << arr->get(i) << " ";
    }
    std::cout << std::endl;

    // Забыли delete arr - УТЕЧКА!
}

void stringOperations() {
    char* buffer = new char[50];
    strcpy(buffer, "Hello"); // ОК

    // Опасная операция
    strcat(buffer, ", this is a very long string that might n

```

```
std::cout << "Buffer: " << buffer << std::endl;

delete[] buffer; // Освобождаем
}

int main() {
    std::cout << "=== Пример 7: Программа с ошибками ===\n" <<

    // 1. Работа с массивом (утечка)
    processArray();

    // 2. Работа со строками
    stringOperations();

    // 3. Дополнительная утечка
    int* extra = new int(42);
    std::cout << "Extra value: " << *extra << std::endl;
    // Забыли delete extra

    // 4. Доступ к неинициализированной памяти
    int* uninit = new int;
    std::cout << "Uninitialized: " << *uninit << std::endl;
    delete uninit;

    std::cout << "\nПрограмма завершена" << std::endl;
    return 0;
}
```

5.2 Пошаговая отладка с Valgrind

Шаг 1: Компиляция

```
g++ -g -o example7 example7_buggy.cpp
```

Шаг 2: Запуск Valgrind

```
valgrind --leak-check=full --show-leak-kinds=all --track-orig
```



Шаг 3: Анализ ошибок

Valgrind покажет:

1. **Invalid write** - выход за границы массива в `fill()`
2. **Uninitialized value** - чтение неинициализированной памяти
3. **Memory leak** - утечка объекта `DynamicArray`
4. **Memory leak** - утечка `int* extra`

5.3 Исправленная версия программы

```
// Сохраните как example7_fixed.cpp
#include <iostream>
#include <cstring>
#include <stdexcept>

class DynamicArray {
private:
    int* data;
    int size;

public:
    DynamicArray(int sz) : size(sz) {
        if(sz <= 0) {
            throw std::invalid_argument("Size must be positive");
        }
        data = new int[size](); // Скобки () инициализируют 1
    }

    ~DynamicArray() {
        delete[] data;
    }

    // Запрещаем копирование (правило трех)
    DynamicArray(const DynamicArray&) = delete;
    DynamicArray& operator=(const DynamicArray&) = delete;

    void fill(int value) {
        for(int i = 0; i < size; i++) { // Исправили условие
            data[i] = value;
        }
    }
}
```

```
int get(int index) const {
    if(index < 0 || index >= size) {
        throw std::out_of_range("Index out of range");
    }
    return data[index];
}

};

void processArray() {
    // Используем умный указатель для автоматического управле
    auto arr = std::make_unique<DynamicArray>(5);
    arr->fill(10);

    for(int i = 0; i < 5; i++) {
        std::cout << arr->get(i) << " ";
    }
    std::cout << std::endl;
    // Автоматически освободится при выходе из функции
}

void stringOperations() {
    // Используем std::string вместо C-строк
    std::string buffer = "Hello";
    buffer += ", this is a safe concatenation!";

    std::cout << "Buffer: " << buffer << std::endl;
    // Память управляется автоматически
}

int main() {
    std::cout << "=== Пример 7: Исправленная версия ===\n" <<

    try {
        processArray();
        stringOperations();

        // Умный указатель вместо raw pointer
        auto extra = std::make_unique<int>(42);
        std::cout << "Extra value: " << *extra << std::endl;

        // Инициализируем память
        auto initialized = std::make_unique<int>(0);
```

```

        std::cout << "Initialized: " << *initialized << std::endl;
    } catch(const std::exception& e) {
        std::cerr << "Error: " << e.what() << std::endl;
        return 1;
    }

    std::cout << "\nПрограмма завершена без ошибок!" << std::endl;
    return 0;
}

```

Проверка исправленной версии:

```

g++ -g -o example7_fixed example7_fixed.cpp
valgrind --leak-check=full ./example7_fixed

```

Часть 6: Продвинутое использование Valgrind

6.1 Massif - профилировщик памяти

Massif показывает использование памяти во времени:

```

// Сохраните как example8_massif.cpp
#include <iostream>
#include <vector>
#include <chrono>
#include <thread>

void allocateMemory() {
    std::vector<int*> chunks;

    for(int i = 0; i < 10; i++) {
        int* chunk = new int[10000]; // 40KB каждый
        chunks.push_back(chunk);

        // Используем память
        for(int j = 0; j < 10000; j++) {
            chunk[j] = j;
        }
    }
}

```

```

        std::cout << "Выделен блок #" << i << std::endl;
        std::this_thread::sleep_for(std::chrono::milliseconds
    }

    // Освобождаем только половину
    for(int i = 0; i < 5; i++) {
        delete[] chunks[i];
    }
    // Вторая половина - утечка!
}

int main() {
    std::cout << "=== Пример 8: Профилирование памяти с Massif

    allocateMemory();

    // Еще немного памяти
    char* buffer = new char[1024 * 1024]; // 1MB
    std::cout << "Выделен 1MB буфер" << std::endl;

    // Не освобождаем - утечка

    std::cout << "\nЗавершение программы" << std::endl;
    return 0;
}

```

Запуск Massif:

```

g++ -g -o example8 example8_massif.cpp
valgrind --tool=massif ./example8

```

Анализ результатов:

```

# Massif создает файл massif.out.XXXXXX
# Конвертируем в читаемый формат
ms_print massif.out.12345 > massif_analysis.txt

# Или используем GUI-инструмент

```



```
sudo apt install massif-visualizer
massif-visualizer massif.out.12345
```

6.2 Cachegrind - профилировщик кэша

Cachegrind анализирует использование кэша процессора:

```
// Сохраните как example9_cache.cpp
#include <iostream>
#include <vector>

const int SIZE = 10000;

void cacheInefficient() {
    std::vector<std::vector<int>> matrix(SIZE, std::vector<int>
    // ПЛОХО: доступ по столбцам (неэффективно для кэша)
    for(int j = 0; j < SIZE; j++) {
        for(int i = 0; i < SIZE; i++) {
            matrix[i][j] = i + j;
        }
    }
}

void cacheEfficient() {
    std::vector<std::vector<int>> matrix(SIZE, std::vector<int>
    // ХОРОШО: доступ по строкам (эффективно для кэша)
    for(int i = 0; i < SIZE; i++) {
        for(int j = 0; j < SIZE; j++) {
            matrix[i][j] = i + j;
        }
    }
}

int main() {
    std::cout << "=== Пример 9: Анализ кэша с Cachegrind ===\n";

    std::cout << "Запуск неэффективного кода..." << std::endl;
    cacheInefficient();
}
```

```

std::cout << "Запуск эффективного кода..." << std::endl;
cacheEfficient();

std::cout << "\nГотово!" << std::endl;
return 0;
}

```

Запуск Cachegrind:

```

g++ -g -o example9 example9_cache.cpp
valgrind --tool=cachegrind ./example9

```

Анализ результатов:

```

# Генерируем отчет
cg_annotate cachegrind.out.12345 --auto=yes > cache_analysis.

# Или используем KCachegrind (GUI)
sudo apt install kcachegrind
kcachegrind cachegrind.out.12345

```



Часть 7: Реальный пример с OpenSSL

7.1 Пример с утечками в реальной библиотеке

```

// Сохраните как example10_openssl.cpp
#include <iostream>
#include <openssl/evp.h>
#include <openssl/err.h>
#include <cstring>

void calculateHash(const char* data) {
    EVP_MD_CTX* mdctx = EVP_MD_CTX_new(); // Выделение
    if(mdctx == nullptr) {
        std::cerr << "Ошибка создания контекста" << std::endl;
        return;
    }
}

```

```

// Инициализация контекста для SHA256
if(1 != EVP_DigestInit_ex(mdctx, EVP_sha256(), nullptr)) {
    std::cerr << "Ошибка инициализации" << std::endl;
    EVP_MD_CTX_free(mdctx); // Важно освободить при ошибке
    return;
}

// Добавление данных
if(1 != EVP_DigestUpdate(mdctx, data, strlen(data))) {
    std::cerr << "Ошибка добавления данных" << std::endl;
    EVP_MD_CTX_free(mdctx);
    return;
}

unsigned char hash[EVP_MAX_MD_SIZE];
unsigned int hash_len;

// Получение хеша
if(1 != EVP_DigestFinal_ex(mdctx, hash, &hash_len)) {
    std::cerr << "Ошибка получения хеша" << std::endl;
    EVP_MD_CTX_free(mdctx);
    return;
}

// Вывод хеша
std::cout << "Хеш SHA256: ";
for(unsigned int i = 0; i < hash_len; i++) {
    printf("%02x", hash[i]);
}
std::cout << std::endl;

// КРИТИЧЕСКАЯ ОШИБКА: забыли освободить контекст!
// EVP_MD_CTX_free(mdctx); // Раскомментируйте эту строку
}

int main() {
    std::cout << "=== Пример 10: Утечка в OpenSSL ===\n" << std::endl;

    // Инициализация OpenSSL
    OpenSSL_add_all_digests();

    // Несколько вызовов с утечками

```

```

for(int i = 0; i < 5; i++) {
    std::cout << "Итерация #" << i << ": ";
    calculateHash("Hello, World!");
}

// Очистка OpenSSL
EVP_cleanup();

std::cout << "\nЗавершение с утечками памяти!" << std::endl;
return 0;
}

```

Компиляция и запуск:

```

# Установите OpenSSL если нужно
sudo apt install libssl-dev

# Компиляция
g++ -g -o example10 example10_openssl.cpp -lssl -lcrypto

# Запуск Valgrind
valgrind --leak-check=full --show-leak-kinds=all ./example10

```

Часть 8: Интеграция Valgrind с CMake

8.1 CMakeLists.txt с поддержкой Valgrind

```

# Сохраните как CMakeLists.txt
cmake_minimum_required(VERSION 3.10)
project(MemoryAnalysisExamples)

set(CMAKE_CXX_STANDARD 11)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# Флаги для отладки
set(CMAKE_CXX_FLAGS_DEBUG "${CMAKE_CXX_FLAGS_DEBUG} -g -O0")

# Добавляем цели

```

```

add_executable(example1 example1_leak.cpp)
add_executable(example2 example2_multiple_leaks.cpp)
add_executable(example3 example3_analyze.cpp)
add_executable(example4 example4_double_free.cpp)
add_executable(example5 example5_use_after_free.cpp)
add_executable(example6 example6_uninitialized.cpp)
add_executable(example7_buggy example7_buggy.cpp)
add_executable(example7_fixed example7_fixed.cpp)
add_executable(example8 example8_massif.cpp)
add_executable(example9 example9_cache.cpp)
add_executable(example10 example10_openssl.cpp)

# Находим OpenSSL
find_package(OpenSSL REQUIRED)
target_link_libraries(example10 OpenSSL::SSL OpenSSL::Crypto)

# Создаем цели для Valgrind
add_custom_target(valgrind_example1
    COMMAND valgrind --leak-check=full ./example1
    DEPENDS example1
)

add_custom_target(valgrind_all
    COMMAND echo "Запуск всех тестов Valgrind..."
    COMMAND valgrind --leak-check=full ./example1
    COMMAND valgrind --leak-check=full ./example2
    COMMAND valgrind --leak-check=full ./example3
    COMMAND valgrind ./example4
    COMMAND valgrind ./example5
    COMMAND valgrind --track-origins=yes ./example6
    DEPENDS example1 example2 example3 example4 example5 exam
)

```

8.2 Скрипт для автоматического тестирования

```

#!/bin/bash
# Сохраните как run_valgrind_tests.sh
# Установите права на выполнение: chmod +x run_valgrind_tests

echo "=== Запуск всех тестов Valgrind ==="
echo ""

```

```

# Создаем папку для результатов
mkdir -p valgrind_reports

# Массив примеров
examples=("example1" "example2" "example3" "example4" "example5" "example6" "example7_buggy" "example7_fixed" "example8" "example9")

for exe in "${examples[@]"; do
    if [ -f "$exe" ]; then
        echo "=====
        echo "Тестирование: $exe"
        echo "=====

        # Запускаем Valgrind
        valgrind --leak-check=full \
            --show-leak-kinds=all \
            --log-file="valgrind_reports/${exe}.log" \
            "$exe" > /dev/null 2>&1

        # Анализируем результаты
        leaks=$(grep -c "definitely lost" "valgrind_reports/${exe}.log")
        errors=$(grep -c "ERROR SUMMARY" "valgrind_reports/${exe}.log")

        if [ "$leaks" -gt 0 ] || [ "$errors" -gt 1 ]; then
            echo "❌ Обнаружены проблемы!"
            echo "    Утечек: $leaks"
            echo "    Ошибок: $errors"
            echo "    Подробности в: valgrind_reports/${exe}.log"
        else
            echo "✅ Тест пройден успешно"
        fi

        echo ""
    else
        echo "⚠️ Файл $exe не найден"
    fi
done

echo "=== Все тесты завершены ==="
echo "Отчеты сохранены в папке valgrind_reports/"

```

Часть 9: Полезные скрипты и утилиты

9.1 Скрипт для поиска проблем с памятью в проекте

```
#!/usr/bin/env python3
# Сохраните как find_memory_issues.py
import subprocess
import os
import sys
import re

def run_valgrind(executable_path):
    """Запускает Valgrind для указанного исполняемого файла"""
    print(f"\n{'='*60}")
    print(f"Анализ: {os.path.basename(executable_path)}")
    print(f"{'='*60}")

    # Команда Valgrind
    cmd = [
        'valgrind',
        '--leak-check=full',
        '--show-leak-kinds=all',
        '--track-origins=yes',
        '--error-exitcode=1',
        executable_path
    ]

    try:
        result = subprocess.run(
            cmd,
            capture_output=True,
            text=True,
            timeout=10 # Таймаут 10 секунд
        )

        # Анализ вывода
        analyze_valgrind_output(result.stdout, result.stderr)

        return result.returncode == 0

    except subprocess.TimeoutExpired:
```

```

        print("🕒 Таймаут: программа выполняется слишком дол
        return False
except Exception as e:
    print(f"❌ Ошибка: {e}")
    return False

def analyze_valgrind_output(stdout, stderr):
    """Анализирует вывод Valgrind"""

    # Ищем утечки памяти
    leak_patterns = {
        'definitely lost': r'definitely lost: (\d+,\d*) bytes
        'indirectly lost': r'indirectly lost: (\d+,\d*) bytes
        'possibly lost': r'possibly lost: (\d+,\d*) bytes in
        'still reachable': r'still reachable: (\d+,\d*) bytes

    }

    print("\n📊 Статистика утечек памяти:")
    print("-" * 40)

    for issue_type, pattern in leak_patterns.items():
        match = re.search(pattern, stderr)
        if match:
            bytes_lost = match.group(1)
            blocks_lost = match.group(2)
            print(f"{issue_type:20} {bytes_lost:>10} bytes в

    # Ищем ошибки
    error_patterns = [
        (r'Invalid read', '📖 Неправильное чтение из памяти')
        (r'Invalid write', '✎ Неправильная запись в память')
        (r'Invalid free', '🗑 Неправильное освобождение памяти')
        (r'Use of uninitialised value', '❓ Использование неинициализированного значения')
        (r'Conditional jump or move depends on uninitialised value', '❓ Зависимость от неинициализированного значения')
    ]

    print("\n🔍 Обнаруженные ошибки:")
    print("-" * 40)

    errors_found = False
    for pattern, description in error_patterns:
        if re.search(pattern, stderr):
            print(f"• {description}")

```



```

        errors_found = True

if not errors_found:
    print("• Ошибок не обнаружено ✅")

# Ищем места выделения памяти для утечек
leak_sources = re.findall(r'by 0x[0-9A-F]+: (.+) \(([^\:]+)'
if leak_sources:
    print("\n🔴 Источники утечек памяти:")
    print("-" * 40)
    seen = set()
    for func, file, line in leak_sources[:5]: # Показывае
        key = f"{file}:{line}:{func}"
        if key not in seen:
            print(f"• {func} (в {file}:{line})")
            seen.add(key)

def find_executables(directory):
    """Находит все исполняемые файлы в директории"""
    executables = []

    for root, dirs, files in os.walk(directory):
        for file in files:
            filepath = os.path.join(root, file)

            # Проверяем, является ли файл исполняемым
            if os.access(filepath, os.X_OK):
                # Игнорируем .so, .a и другие библиотеки
                if not filepath.endswith(('.so', '.a', '.dll',
                    executables.append(filepath)

    return executables

def main():
    if len(sys.argv) > 1:
        target = sys.argv[1]
    else:
        target = '.' # Текущая директория

    if not os.path.exists(target):
        print(f"❌ Ошибка: путь '{target}' не существует")
        sys.exit(1)

```

```
print("🔍 Поиск исполняемых файлов...")

if os.path.isfile(target) and os.access(target, os.X_OK):
    executables = [target]
else:
    executables = find_executables(target)

if not executables:
    print("❌ Исполняемые файлы не найдены")
    sys.exit(1)

print(f"Найдено {len(executables)} файлов для анализа")

results = []
for exe in executables:
    success = run_valgrind(exe)
    results.append((exe, success))

# ИТОГОВЫЙ ОТЧЕТ
print(f"\n{'='*60}")
print("📋 ИТОГОВЫЙ ОТЧЕТ")
print(f"{'='*60}")

passed = sum(1 for _, success in results if success)
failed = len(results) - passed

print(f"Всего проанализировано: {len(results)} файлов")
print(f"✅ Успешно: {passed}")
print(f"❌ С ошибками: {failed}")

if failed > 0:
    print("\nФайлы с ошибками:")
    for exe, success in results:
        if not success:
            print(f"    • {exe}")

return 0 if failed == 0 else 1

if __name__ == "__main__":
    sys.exit(main())
```

Часть 10: Практические задания для закрепления

Задание 1: Найти и исправить утечки

```
// Сохраните как exercise1.cpp
#include <iostream>
#include <vector>

class Resource {
private:
    int* data;
    int size;

public:
    Resource(int sz) : size(sz) {
        data = new int[size];
        std::cout << "Resource created with size " << size <<
    }

    // TODO: Добавить деструктор, конструктор копирования, оп

    void process() {
        for(int i = 0; i < size; i++) {
            data[i] = i * 10;
        }
    }

    void print() const {
        std::cout << "Resource data: ";
        for(int i = 0; i < size; i++) {
            std::cout << data[i] << " ";
        }
        std::cout << std::endl;
    }
};

void createResources() {
    std::vector<Resource*> resources;

    for(int i = 1; i <= 5; i++) {
        Resource* res = new Resource(i * 10);
```

```

        res->process();
        resources.push_back(res);
    }

    // TODO: Освободить все ресурсы
}

int main() {
    std::cout << "=== Задание 1: Утечки памяти ===\n" << std::endl;

    createResources();

    // TODO: Исправить утечку
    Resource* single = new Resource(100);
    single->print();

    std::cout << "\nПрограмма завершена" << std::endl;

    return 0;
}

```

Задание 2: Отладить некорректное использование памяти

```

// Сохраните как exercise2.cpp
#include <iostream>
#include <cstring>

class StringWrapper {
private:
    char* str;
    int length;

public:
    StringWrapper(const char* s) {
        length = strlen(s);
        str = new char[length]; // TODO: Исправить - нужно +1
        strcpy(str, s); // Опасная операция!
        std::cout << "String created: " << str << std::endl;
    }

    ~StringWrapper() {

```

```

        delete[] str;
    }

    // TODO: Запретить копирование или реализовать правильно

void append(const char* s) {
    int new_length = length + strlen(s);
    char* new_str = new char[new_length]; // TODO: Исправить

    strcpy(new_str, str);
    strcat(new_str, s); // Опасная операция!

    delete[] str;
    str = new_str;
    length = new_length;

    std::cout << "After append: " << str << std::endl;
}

};

void testStrings() {
    StringWrapper s1("Hello");
    StringWrapper s2 = s1; // TODO: Проблема - поверхностное копирование

    s1.append(", World!");
    s2.append(" from copy!"); // ОПАСНО: двойное удаление!
}

int main() {
    std::cout << "=== Задание 2: Некорректное использование памяти ===\n";

    testStrings();

    // Дополнительные проблемы
    char buffer[10];
    strcpy(buffer, "Testing buffer overflow potential"); // Опасная операция!

    std::cout << "\nПрограмма завершена" << std::endl;

    return 0;
}

```

Заключение

Ключевые команды Valgrind:

```
# Основные команды
valgrind --leak-check=full ./program           # Проверка утечек
valgrind --track-origins=yes ./program         # Отслеживание н
valgrind --tool=massif ./program               # Профилирование
valgrind --tool=cachegrind ./program           # Профилирование

# Полезные флаги
--show-leak-kinds=all          # Показать все типы утечек
--errors-for-leak-kinds=all    # Считать все утечки ошибками
--suppressions=file.supp      # Игнорировать определенные ошибки
--xml=yes --xml-file=output.xml # Вывод в XML формате
```



Лучшие практики:

1. **Всегда компилируйте с -g** для отладочной информации
2. **Используйте Valgrind во время разработки**, а не только для отладки
3. **Интегрируйте Valgrind в CI/CD** пайплайн
4. **Создавайте suppression-файлы** для известных ложных срабатываний
5. **Регулярно проверяйте** свои программы на утечки памяти

Дополнительные инструменты:

```
# AddressSanitizer (более быстрый, но менее точный)
g++ -g -fsanitize=address -o program program.cpp
./program

# LeakSanitizer (часть AddressSanitizer)
g++ -g -fsanitize=leak -o program program.cpp
./program

# UndefinedBehaviorSanitizer
```

```
g++ -g -fsanitize=undefined -o program program.cpp  
./program
```

Этот подробный гайд покрывает все аспекты работы с Valgrind для начинающих. Практикуйтесь на примерах, и вы быстро освоите этот мощный инструмент!